PRN:

Name:

Experiment no 5

Title:Task-Level Parallelism using OpenMP Sections and Tasks

Objective:

To understand and implement task-level parallelism using OpenMP sections and task constructs, and to analyze their suitability and performance for applications containing independent, irregular, and dynamically generated tasks.

Why?

So far, we have used OpenMP mainly to parallelize **loops**. However, not every application can be naturally expressed as a parallel loop.

Consider the following situations:

- Two independent functions need to execute simultaneously.
- A program contains different operations with different execution times.
- The amount of work is not known in advance.
- An algorithm generates new work dynamically.
- A recursive algorithm creates smaller subproblems.
- Some computations can start only after other computations finish.

In such cases, task-level parallelism becomes useful.

OpenMP provides:

- sections : suitable for a small, fixed number of independent tasks.
- task : suitable for dynamically generated or irregular tasks.

HPC in Practice

Task-based parallelism is used in several real-world applications.

Domain	**Example**
AI / Machine Learning	Parallel preprocessing and inference tasks
Computer Vision	Independent image-processing operations
Scientific Computing	Independent simulation components
Graph Processing	Irregular graph traversal

Domain	Example
Robotics	Parallel sensor-processing tasks
Multimedia	Audio/video processing pipelines
Algorithms	Recursive sorting and divide-and-conquer algorithms
Compilers	Parallel compilation of independent modules

Suppose an AI application has the following independent operations:

1. Image preprocessing
2. Feature extraction
3. Data normalization
4. Metadata processing

Would a parallel for be the most natural way to express this parallelism?

If the operations are different and have different execution times, task-level parallelism may be more appropriate.

Answer the following before starting the implementation.

1. What is the difference between data parallelism and task parallelism?
2. When is parallel for more suitable than task?
3. What is the purpose of the OpenMP sections construct?
4. What is the purpose of the OpenMP task construct?
5. Why can task creation introduce overhead?
6. Give two real-world applications where task-level parallelism may be preferable to loop parallelism.

Section A

Task 1: Independent Tasks using OpenMP Sections

Consider the following four independent operations:

1. Calculate the sum of an array.
2. Calculate the maximum element.
3. Calculate the minimum element.
4. Calculate the average of the array.

Implement:

Part A – Sequential Version

Execute all four operations sequentially.

Part B – OpenMP Version

Use:

#pragma omp parallel sections

and divide the four operations into separate sections.

Each section should perform one operation.

Requirements

1. Execute the program sequentially.
2. Execute the program using OpenMP sections.
3. Verify that both versions produce identical results.
4. Measure execution time.

Prepare:

Version	**Execution Time**
Sequential	
OpenMP Sections	

Calculate: Speedup

Lmao2

Analysis

Answer:

1. Why can these four operations execute concurrently?
2. Why is sections suitable for this problem?
3. What happens if the number of sections is smaller than the number of threads?
4. Will increasing the number of threads always improve performance?

Task 2: Understanding the Difference Between Sections and Parallel For

Implement the following problem using two approaches.

Problem

Given an array of N elements, perform the following independent operations:

- Sum
- Maximum
- Minimum
- Count of even numbers

Implementation 1

Use OpenMP parallel for.

Implementation 2

Use OpenMP parallel sections.

Compare the two approaches in terms of:

- Parallelization strategy
- Number of threads
- Work distribution
- Execution time
- Scalability
- Suitability for the problem

Prepare the following table:

Feature	**parallel for**	**parallel sections**
Type of parallelism		
Work distribution		
Suitable for		
Scalability		
Overhead		

Section B

Task 3: Parallel Pipeline using OpenMP Tasks

Consider a simplified data-processing pipeline:

Input → Preprocessing → Computation → Postprocessing

Implement the following operations:

Stage 1: Preprocessing

Perform an operation such as normalization or filtering.

Stage 2: Computation

Perform a computationally intensive operation such as vector transformation.

Stage 3: Postprocessing

Calculate statistics such as sum, minimum, maximum, or average.

Implement the operations using OpenMP tasks.

Use:

#pragma omp parallel

#pragma omp single

#pragma omp task

Requirements

1. Implement the sequential version.
2. Implement the task-based version.
3. Measure execution time.
4. Verify correctness.
5. Explain how the tasks are created and executed.

Task 4: Independent Matrix Operations using OpenMP Sections

Given matrices A and B, perform the following independent computations:

- Matrix addition
- Matrix subtraction
- Transpose of A
- Transpose of B

Use OpenMP sections to execute the four operations concurrently.

Compare:

- Sequential implementation
- OpenMP sections implementation

Test with at least three matrix sizes.

Suggested sizes:

- 500 × 500
- 1000 × 1000
- 2000 × 2000

Prepare:

Matrix Size	Sequential Time	Sections Time	Speedup
500 × 500			
1000 × 1000			
2000 × 2000			

Think Like an HPC Programmer

The four matrix operations may not require the same amount of computation.

Answer:

1. What happens if one section takes significantly longer than the others?
2. Will all threads remain equally busy?
3. What is meant by load imbalance?
4. How can task-based parallelism help with irregular workloads?
5. Why might sections become inefficient when the number of independent tasks increases?

Section C

Challenge: Recursive Task Parallelism

Implement **Merge Sort** using OpenMP tasks.

Requirements

1. Implement sequential Merge Sort.
2. Implement parallel Merge Sort using OpenMP task.
3. Divide the array recursively.
4. Create separate tasks for the left and right subarrays.
5. Use an appropriate synchronization mechanism before merging.

You may use:

#pragma omp task

and

#pragma omp taskwait

Performance Experiment

Run the program with:

- 1 thread
- 2 threads
- 4 threads
- 8 threads
- Maximum available threads

Test with large input sizes.

Prepare:

Threads Execution Time Speedup Efficiency

1

2

4

8

Maximum

Challenge Analysis

Investigate the effect of task granularity.

Run the program with different task creation thresholds.

For example:

- Create tasks until subarray size = 10
- 100
- 1000
- 10,000

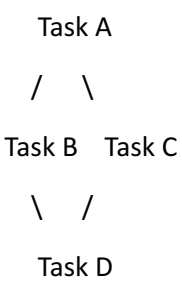
Compare the execution time.

Answer:

1. Why does creating a task for every small subproblem reduce performance?
2. What is task creation overhead?
3. Why is taskwait required?
4. What happens when the number of tasks becomes much larger than the number of threads?
5. What is the relationship between task granularity and performance?

Advanced Exploration: Task Dependencies

Consider the following computation graph:



Task D can start only after Tasks B and C are completed.

Investigate how OpenMP task dependencies can represent such relationships.

Explore:

#pragma omp task depend(out: x)

and

#pragma omp task depend(in: x)

Explain how task dependencies differ from simply using taskwait.

Performance Analysis

For at least one task-based implementation, analyze:

1. Execution Time

2. Speedup

3. Efficiency

4. Scalability

Study how execution time changes as the number of threads increases.

HPC / AI Case Study

Choose **one** of the following applications:

- AI model preprocessing
- Neural network inference pipeline
- Computer vision pipeline
- Robotic sensor processing
- Scientific simulation
- Graph analytics
- Parallel file/data processing

Identify:

1. What are the independent tasks?
2. Which tasks can execute concurrently?
3. Would you use sections, task, or parallel for?
4. Are there dependencies between tasks?
5. What could cause load imbalance?
6. How could task-based parallelism improve performance?

Write a 300–500 word case study.

Answer the following:

1. Differentiate between parallel for, sections, and task.
2. When should sections be preferred over parallel for?
3. What is task-level parallelism?
4. What is the role of single when creating OpenMP tasks?
5. Why can task-based execution handle irregular workloads better than static loop parallelism?
6. What is task creation overhead?
7. Explain load imbalance in task-based applications.
8. Why is synchronization necessary when tasks have dependencies?
9. Explain how recursive algorithms can be parallelized using OpenMP tasks.
10. Why can creating too many tasks reduce performance?
11. How does task granularity affect scalability?
12. Compare sections and tasks for a problem containing 4 fixed independent operations and a problem containing thousands of dynamically generated operations.

After completing the assignment, answer:

1. Which parallelization method performed best for your selected application?
2. What was the major source of parallel overhead?
3. Did increasing the number of threads always improve performance? Why?
4. Which type of problem would you choose for:
    - parallel for
    - sections

- task

5. Identify one application from your area of interest where task-level parallelism could be useful.

Deliverables

Students must submit in an pdf:

1. Source code for all completed tasks.
2. Sequential and parallel implementations where applicable.
3. Execution results and snapshots.
4. Performance tables.
5. Speedup and efficiency calculations.
6. Required graphs.
7. Answers to discussion questions.
8. AI/HPC case study.
9. Brief conclusion highlighting the suitability of sections and tasks.